Geospatial Web Services

Lecture 6

Web Service Type

- The communication between a web service and a client involves the client sending request to the web service, and the web service returning response to the client

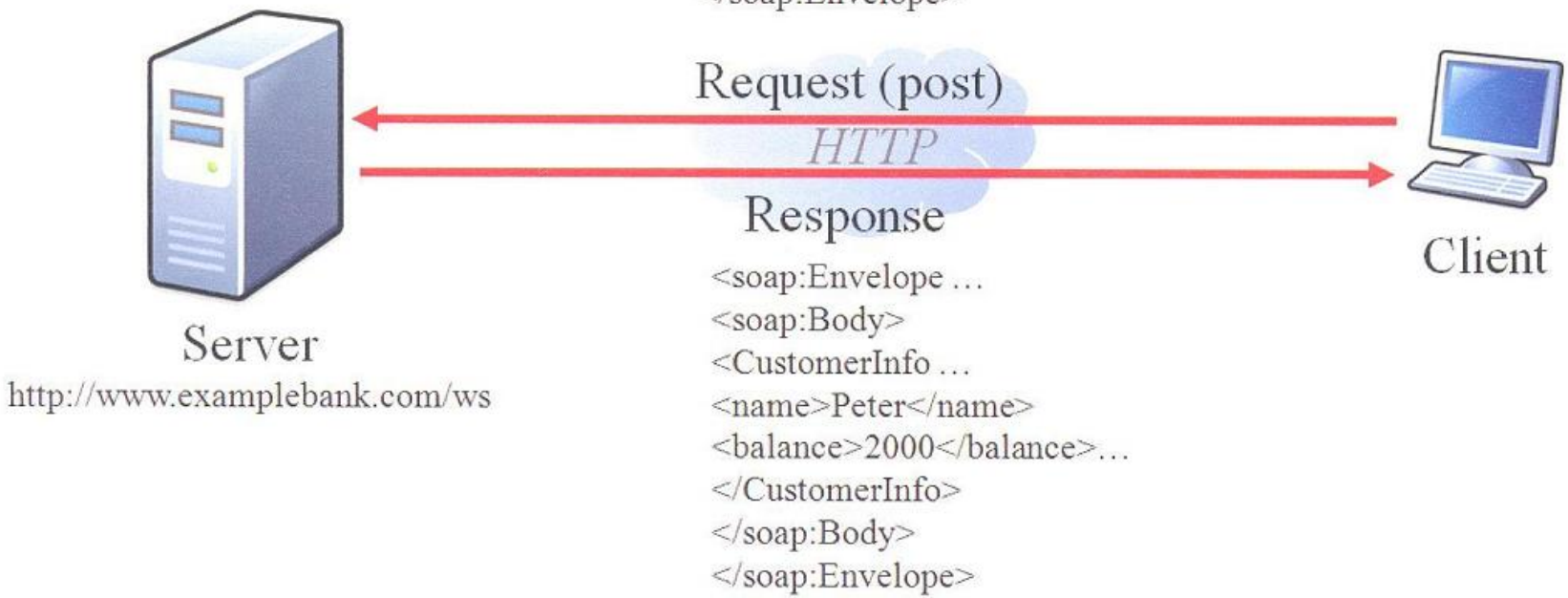
Web Service Type

- Depending on the format of communication used, the web service can be categorized into two main types
  - SOAP-based web services
  - REST-style web services

SOAP based web services

- SOAP (Simple Object Access Protocol) is a protocol specification for exchanging structured information in XML format.

"SOAP based web services use HTTP post to send request. The requests and responses are in SOAP encapsulated XMLs"


<soap:Envelope …
<soap:Body>
<GetInfo …
<Customer>2</Customer>
</GetInfo>
</soap:Body>
</soap:Envelope>
Request (post)
HTTP
Response
<soap:Envelope …
<soap:Body>
<CustomerInfo …
<name>Peter</name>
<balance>2000</balance>…
</CustomerInfo>
</soap:Body>
</soap:Envelope>
Server
http://www.examplebank.com/ws
Client

REST-style Web Services

- REST (Representational State Transfer)

"RESTful web services are web services that transmit data over HTTP without any additional messaging layer such as SOAP."

- In the most common RESTful web service architecture, the client sends all parameters in the request URL



Server

Request: parameters in the URL, e.g.,
http://www.examplebank.com/Customers/1

HTTP



Client

Response: any structured format, such as JSON or XML (without SOAP), e.g.,

{"name": "Peter",
"balance": 2000}

RESTful Web Services Principles

- Identification of resources through URI or URLs
  - Everything is a resource and every resource gets a URI.
  - The resources and their URLs are organized in a hierarchy that is intuitive, predictable and easy to understand

RESTful Web Services Principles

The ArcGIS Server Web services provide a hierarchy of resources. For example, the URL for all Web services available on an ArcGIS server is

```
http: //server.arcgisonline.com/ArcGIS/rest/services
```

The URL for a map service named "ESRI_StreetMap_World_2D" is the above URL, plus the service name and the service type (e.g., MapServer).

```
http: //server.arcgisonline.com/ArcGIS/rest/services/
ESRI_StreetMap_World_2D/MapServer
```

The URL to identify the first data layer in this map service is the above URL, plus "0" (the layer count starts from 0).

```
http: //server.arcgisonline.com/ArcGIS/rest/services/
ESRI_StreetMap_World_2D/MapServer/0
```

RESTful Web Services Principles

- Manipulation of resources through their representations
  - A resource can be returned in many formats e.g. JSON, XML etc.
  - A resource can have multiple representations
  - RESTful web services allow clients to indicate which representation of a resource they wish to receive

ArcGIS map services can respond in several representations. Using the following ArcGIS Online map service as an example,

`http://sampleserver1.arcgisonline.com/ArcGIS/rest/services/Portland/ESRI_LandBase_WebMercator/MapServer`

The response can be in the following formats according to the parameter "f" in the request URL:

- f=html: The response is an HTML page.
- f=json: The response is a JSON object. This format is used primarily by ArcGIS APIs for JavaScript, Flex, and Silverlight (see chapter 4).
- f=help: The response is a context-sensitive Help document that provides information about the resource.
- f=lyr: The response is a layer file for viewing in ArcMap.
- f=nmf: The response is a layer file for viewing in ArcGIS Explorer Desktop.
- f=jsapi: The response is a map viewer using the ArcGIS API for JavaScript. This map viewer allows you to dynamically interact with the map service.
- f=ve: The response is a Web page for viewing the map service interactively in a Web browser using Microsoft Bing Maps.
- f=gmaps: The response is a Web page for viewing the map service interactively in a Web browser using Google Maps.
- f=kmz: The response is a KML document wrapped in a KMZ file. It may be a footprint— that is, the spatial extent of a Web service—or the result of an operation.

RESTful Web Services Principles

- REST also supports CRUD (create, read, update and delete) operations by using different HTTP methods
  - HTTP **Post**: Create a resource
  - HTTP **Get**: read a resource
  - HTTP **Put**: Update or create a resource
  - HTTP **Delete**: Delete a resource

RESTful Web Services Principles

- Self descriptive representation
  - Each client request and server response is a message
  - RESTful applications expect each message to be self-descriptive

Using the ArcGIS Server REST API to generate a map covering the United States, asking the return to be a JPEG image of 800 pixels by 500 pixels, the request is the following:

http://server.arcgisonline.com/ArcGIS/rest/services/ESRI_ShadedRelief_World_2D/MapServer/export?

bbox=-185.33, 15.20, -9.53, 74.08

&size=800, 500

&format=jpg

&dpi=96

&f=image

Using the ArcGIS Server REST API to query a map service to obtain the median household income of every county in California, asking the return to be in JSON format, the request is the following:

http://server.arcgisonline.com/ArcGIS/rest/services/Demographics/ESRI_MedianIncome_US_2D/MapServer/0/query?

where=STATE_NAME='California'

&outFields=MEDHINC_CY

&f=pjson

Using Google's REST API to search for Web sites related to "GIS" (the default return format is JSON), the request is the following:

http://ajax.googleapis.com/ajax/services/search/web?v=1.0&q=GIS

Using Yahoo's REST API to geocode the address "701 First Ave, Sunnyvale, CA" (the default return format is XML), the request is the following:

http://local.yahooapis.com/MapsService/V1/geocode?

appid=yourApplicationId

&street=701+First+Ave

&city=Sunnyvale

&state=CA

Comparing SOAP and REST

- In many cases, the simplicity and the efficiency of using REST outweighs the rigorous discipline of SOAP and the complexity in introducing SOAP-based web services

Advantages of REST over SOAP

- For **Producers**: Lowering the cost of creating a service and lowering the overhead for hosting and supporting the service
- For **Users**: Lowering the learning curve and reducing the time and money needed to build GIS applications
- For **Managers**: Providing highly desirable architectural properties such as scalability, performance, reliability and extensibility